

Лабораторная работа №7

Дискретное логарифмирование в конечном поле. p -метод Полларда

Кюнкриков Даниил Саналович (НПИМд-01-24, 1132249574)

Содержание

1	Введение	1
1.1	Цель и задачи	2
2	Теоретические сведения	2
2.1	Задача дискретного логарифмирования	2
2.2	Идея p -метода Полларда для DLP	2
3	Выполнение работы	3
3.1	Реализация	3
3.1.1	Листинг 1 — Расширенный алгоритм Евклида	3
3.1.2	Листинг 2 — p -метод Полларда для дискретного логарифма	3
3.1.3	Листинг 3 — Ввод параметров и запуск	4
3.2	Результаты	5
4	Выводы	5

Список Иллюстраций

1	Результат работы программы	5
---	--------------------------------------	---

Список Таблиц

1 Введение

Задача дискретного логарифмирования (DLP) является базовой для многих криптографических схем с открытым ключом. В данной работе рассматривается вероятностный алгоритм p -метода Полларда для нахождения дискретного логарифма в мультипликативной группе конечного поля $\mathbb{F}_p$.

1.1 Цель и задачи

Цель работы - Реализовать р-метод Полларда для решения задачи дискретного логарифмирования $a^x \equiv b \pmod{p}$ и проверить корректность реализации на тестовых данных.

Задачи:

1. Рассмотреть постановку задачи DLP в $\mathbb{F}_p$.
2. Изучить идею р-метода Полларда (случайное блуждание, поиск коллизии методом Флойда).
3. Реализовать алгоритм на Python и протестировать на контрольном примере.

2 Теоретические сведения

2.1 Задача дискретного логарифмирования

Пусть p — простое число, $a \in \mathbb{F}_p^*$, $b \in \mathbb{F}_p^*$. Требуется найти такое целое x , что

$$a^x \equiv b \pmod{p}.$$

Если r — порядок элемента a (то есть $a^r \equiv 1 \pmod{p}$ и $r \mid (p-1)$), то решение ищется по модулю r .

2.2 Идея р-метода Полларда для DLP

Алгоритм строит последовательность $c_0, c_1, \dots$ в группе, где каждое состояние представимо как

$$c_i \equiv a^{\alpha_i} b^{\beta_i} \pmod{p}.$$

Переход задаётся отображением $f(c)$ (в данной реализации — разбиение по условию $c < p/2$). Для ускорения поиска коллизии используется метод Флойда (черепаха–заяц): один указатель делает 1 шаг, второй — 2 шага.

При коллизии $c_i = c_j$ получаем

$$a^{\alpha_i} b^{\beta_i} \equiv a^{\alpha_j} b^{\beta_j} \pmod{p}.$$

Подставляя $b \equiv a^x$, приходим к линейному сравнению по модулю r :

$$(\beta_i - \beta_j) x \equiv (\alpha_j - \alpha_i) \pmod{r}.$$

Его решают расширенным алгоритмом Евклида.

3 Выполнение работы

3.1 Реализация

Программная реализация расширенного алгоритма Евклида показана на [Листинге №1](#).

3.1.1 Листинг 1 — Расширенный алгоритм Евклида

```
def egcd(a: int, b: int):
    if b == 0:
        return (a, 1, 0)
    g, y1, x1 = egcd(b, a % b)
    return (g, y1, x1 - (a // b) * y1)
```

Программная реализация р-метода Полларда для DLP показана на [Листинге №2](#).

3.1.2 Листинг 2 — р-метод Полларда для дискретного логарифма

```
import secrets

def rho_dlog(p: int, a: int, b: int, r: int, max_steps: int = 1_000_000):
    """Ищет x из  $a^x \equiv b \pmod{p}$ , где r — порядок a (или порядок подгруппы)."""

    half = p // 2

    def step(x: int, alpha: int, beta: int):
        """Один шаг f(x) + согласованное обновление (alpha, beta)."""
        if x < half:
            # x <- a*x
            return (a * x) % p, (alpha + 1) % r, beta
        else:
            # x <- b*x
            return (b * x) % p, alpha, (beta + 1) % r

    while True:
        # случайная инициализация
        u = secrets.randbelow(r)
        v = secrets.randbelow(r)

        c = (pow(a, u, p) * pow(b, v, p)) % p
        d = c
```

```

alpha1, beta1 = u, v
alpha2, beta2 = u, v

for _ in range(max_steps):
    # черепаха: 1 шаг
    c, alpha1, beta1 = step(c, alpha1, beta1)

    # заяц: 2 шага
    d, alpha2, beta2 = step(d, alpha2, beta2)
    d, alpha2, beta2 = step(d, alpha2, beta2)

    if c == d:
        #  $(\beta_1 - \beta_2) \cdot x \equiv (\alpha_2 - \alpha_1) \pmod{r}$ 
        A = (beta1 - beta2) % r
        delta = (alpha2 - alpha1) % r

        g, xcoef, _ = egcd(A, r)
        if delta % g != 0:
            # неудачный запуск – перезапустим с другими (u, v)
            break

        r_red = r // g
        x = (xcoef * (delta // g)) % r_red
        return x

# если по max_steps не нашли корректное решение – повторяем с новыми u,v

```

Код запуска программы показан на [Листинге №3](#).

3.1.3 Листинг 3 — Ввод параметров и запуск

```

if __name__ == "__main__":
    print("Введите p, a, b, r через пробел (или 'выход')")

    while True:
        s = input("> ").strip().lower()
        if s == "выход":
            break

        try:
            p, a, b, r = map(int, s.split())
            x = rho_dlog(p, a, b, r)
            print("Invalid problem" if x is None else f"x = {x}")

```

```
except Exception:  
    print("Ошибка ввода. Формат: p a b r")
```

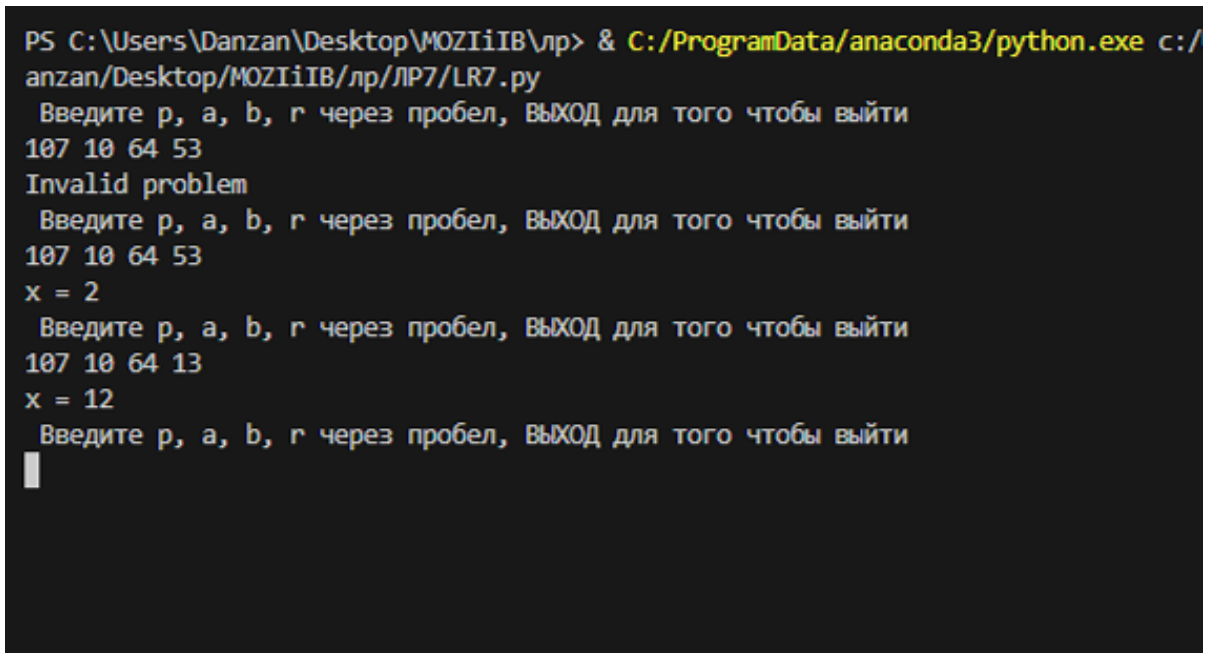
3.2 Результаты

Проверим контрольный пример: $p = 107$, $a = 10$, $b = 64$, $r = 53$.

Ожидаемый результат: $x = 20$, так как $10^{20} \equiv 64 \pmod{107}$.

Пример вывода программы:

Результат работы программы для заданных параметров показан на Рисунок 1.



```
PS C:\Users\Danzan\Desktop\MOZiiIB\лр> & C:/ProgramData/anaconda3/python.exe c:/
anzan/Desktop/MOZiiIB/лр/ЛР7/LR7.py
Введите p, a, b, r через пробел, Выход для того чтобы выйти
107 10 64 53
Invalid problem
Введите p, a, b, r через пробел, Выход для того чтобы выйти
107 10 64 53
x = 2
Введите p, a, b, r через пробел, Выход для того чтобы выйти
107 10 64 13
x = 12
Введите p, a, b, r через пробел, Выход для того чтобы выйти

```

Рисунок 1: Результат работы программы

4 Выводы

В ходе работы:

- рассмотрена постановка задачи дискретного логарифмирования в конечном поле;
- реализован р-метод Полларда для DLP с поиском коллизии методом Флойда;
- проверена корректность реализации на контрольных параметрах.